QUICK REFERENCE

Wire Framework — developer card.

A single-glance reference for the commands, release types, lifecycle steps and skills that ship with Wire v3.4.20. Pin this above your desk, keep it open beside Claude Code, or hand it to a new analytics engineer on day one.

§1 Installation

```
# In Claude Code
/plugin marketplace add rittmananalytics/wire-plugin
/plugin install wire@rittman-analytics
/reload-plugins
/wire:help
```

§2 Starting an engagement

```
/wire:new                    # Create engagement + first release
/wire:start                  # All releases + next actions
/wire:status <release>       # Check one release in detail
/wire:plan [release]         # Plan Mode, agree a session plan
```

§3 Three gates, every artifact

GATE 1

Generate

AI reads the spec and upstream inputs, produces the artifact. Files land in the release folder.

GATE 2

Validate

Automated checks against conventions. **PASS** or **FAIL**, with the specific failures listed.

GATE 3

Review

You or the stakeholder approves. Status moves to `approved` or `changes_requested`.

Each gate updates `status.md` and appends to `execution_log.md`. Nothing happens off the books.

§4 Release types & in-scope artifacts

RELEASE TYPE	KEY ARTIFACTS	USE WHEN
`discovery`	problem_definition · pitch · release_brief · sprint_plan	Uncertain scope — run before delivery.
`full_platform`	All 15 artifacts	SOW to production dashboards, end-to-end.
`pipeline_only`	requirements · pipeline_design · data_model · pipeline · dbt · data_quality · deployment	New pipeline + dbt, no BI layer.
`dbt_development`	requirements · conceptual_model · data_model · dbt · data_quality	dbt layer only, data already in warehouse.
`dashboard_extension`	requirements · mockups · dashboards · uat	New dashboards on an existing semantic layer.
`dashboard_first`	14 artifacts inc. viz_catalog · seed_data · data_refactor	Prototyping with seed data before client data access.
`enablement`	requirements · training · documentation	Training + docs for an existing platform.

§5 Full platform artifact order

PHASE 1 Requirements	`requirements`
PHASE 2 Design	`conceptual_model` → `pipeline_design` → `data_model` → `mockups`
PHASE 3 Development	`pipeline` → `dbt` → `orchestration` → `semantic_layer` → `dashboards`
PHASE 4 Testing	`data_quality` → `uat`
PHASE 5 Deployment	`deployment`
PHASE 6 Enablement	`training` → `documentation`

§6 Discovery release flow

```
/wire:problem-definition-generate <release>
/wire:problem-definition-validate <release>
/wire:problem-definition-review   <release>

/wire:pitch-generate <release>   # 10-section Shape Up pitch
/wire:pitch-validate <release>
/wire:pitch-review   <release>   # Betting table review

/wire:release-brief-generate <release>
/wire:release-brief-validate <release>
/wire:release-brief-review   <release>

/wire:sprint-plan-generate <release>
/wire:sprint-plan-validate <release>
/wire:sprint-plan-review   <release>

/wire:release:spawn <release>    # Create downstream releases
```

§7 Session lifecycle (v3.4.20+)

No explicit session commands. State is managed automatically.

WHAT HAPPENS	HOW
Context loads on first message	`engagement-context` skill fires automatically.
Status updates per command	`status.md` written by each command.
Activity logged	`execution_log.md` appended after each command and skill activation.
Optional structured planning	`/wire:plan [release]` enters Plan Mode and proposes a 3–5 step plan.

> DEPRECATED `session:start` and `session:end` are gone — running them shows a migration notice.

§8 Engagement directory structure

```
.wire/
  engagement/
    context.md          ← client background, stakeholders, working agreements
    sow.md              ← statement of work
    calls/              ← meeting transcripts (add manually)
    org/                ← org charts, RACI
  releases/
    01-<release-name>/
      status.md         ← process state (YAML frontmatter + notes)
      execution_log.md  ← timestamped command and skill activity log
      requirements/     ← generated artifact files
      design/
      development/
      ...
  research/
    sessions/           ← auto-saved technical research summaries
```

§9 Utility commands

```
/wire:utils-run-dbt <release>            # Run generated dbt models
/wire:utils-meeting-context <release>    # Pull Fathom transcript context
/wire:utils-jira-create <release>        # Set up Jira Epic + Tasks
/wire:utils-linear-create <release>      # Set up Linear Project + Issues
/wire:utils-docstore-setup <release>     # Configure Confluence or Notion store
/wire:mcp [list|view|update|auth]        # Manage MCP server connections
/wire:autopilot [sow]                    # Autonomous end-to-end delivery
/wire:migrate                            # Migrate pre-v3.4 repo structure
/wire:archive <release>                  # Archive a completed release
```

§10 Skills that auto-activate

TRIGGER	SKILL	WHAT IT DOES
`.wire/` dir present	`engagement-context`	Loads release state, surfaces research.
dbt models / files	`dbt-development`	Enforces naming, testing, 3-layer conventions.
LookML files	`lookml-content-authoring`	Validates views, explores, dashboards.
Looker MCP available	`lookml-content-authoring (MCP)`	Live schema validation against Looker.
Dagster files	`dagster`	Assets-first patterns, dagster-dbt integration.
Python files	`dignified-python`	Modern type syntax, LBYL, pathlib, Click.
dbt error output	`dbt-troubleshooting`	Classifies and resolves dbt errors.
"visualise lineage"	`dbt-dag`	Generates Mermaid lineage diagrams.
Semantic layer work	`dbt-semantic-layer`	Metric layer patterns and validation.
Unit test files	`dbt-unit-testing`	dbt unit-test structure and coverage.
dbt analytics Q&A	`dbt-analytics-qa`	Answers business questions via semantic layer.
Dashboard mockup request	`looker-dashboard-mockup`	Pixel-accurate HTML Looker mockups.
Technical research	`research`	Saves and surfaces prior research findings.

§11 Troubleshooting

PROBLEM	FIX
Commands not found	Run `/reload-plugins`; if still missing, reinstall.
Poor generate output	Add more source material to `engagement/` and `requirements/`.
Validate keeps failing	Read the error — it names the specific failing check.
Status.md out of sync	Run `/wire:status <release>` to reconcile.
Context skill not firing	Check `.wire/` exists at the repo root.
Autopilot blocked	Fix the blocking artifact manually, re-run `/wire:autopilot`.

§12 Key files to know

FILE	PURPOSE
`USER_GUIDE.md`	Full user guide — release types, examples, FAQ.
`wire/specs/<path>.md`	Workflow spec for each command — edit to change behaviour.
`wire/skills/<name>/SKILL.md`	Skill definition — edit to change when / how it activates.
`wire/scripts/build-packages.sh`	Rebuild dist after changing specs or skills.
`.wire/releases/<id>/status.md`	Current release state — readable + writable.
`.wire/releases/<id>/execution_log.md`	Append-only activity log.